3. The Arduino Bluetooth module at the other end receives the data and sends it to the Arduino through the TX pin of the Bluetooth module (connected to RX pin of Arduino).
4. The code uploaded to the Arduino checks the received data and compares it. If the received data is 1, the LED turns ON.
5. The LED turns OFF when the received data is 0. You can open the serial monitor and watch the received data while connecting.

SOURCE CODE

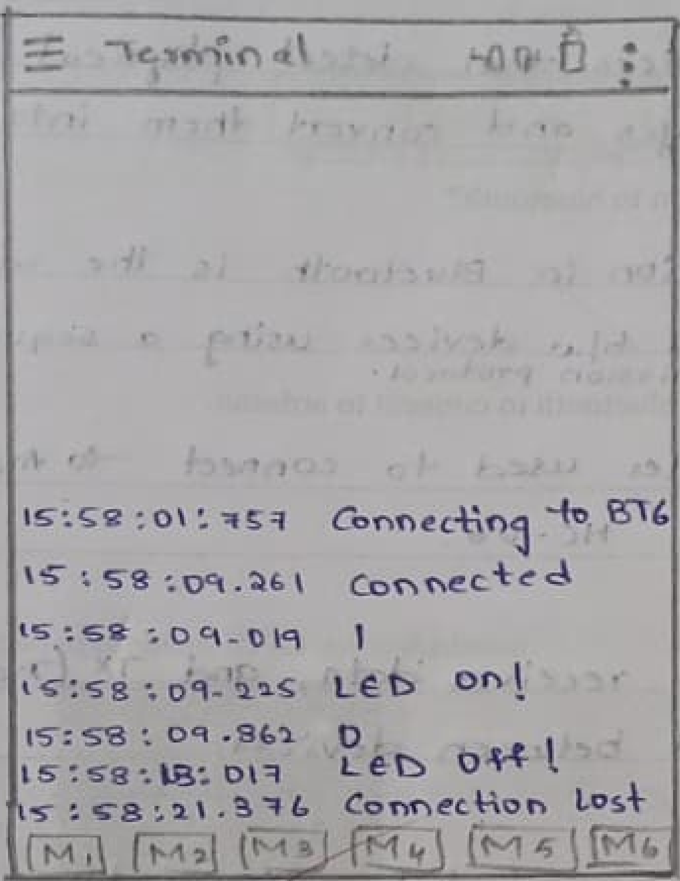
```
#include <SoftwareSerial.h>
SoftwareSerial Bluetooth(8,9);
int LED=13;
int Data;
void setup(){
  Bluetooth.begin(9600);
  Serial.println("Waiting for command...");
  Bluetooth.println("Send 1 to turn on the LED.
                     Send 0 to turn off");
  pinMode(LED, OUTPUT);
}
void loop(){
  if (Bluetooth.available()){
    Data= Bluetooth.read();
    if(Data=='1'){
        digitalWrite(LED,HIGH);
        Serial.println("LED On!");
        Bluetooth.println("LED On!");
  }
```

```
        else if (Data == '0') {
            digitalWrite (LED, LOW);
            Serial.println ("LED Off!");
            Bluetooth.println ("LED Off!");
        }
    }
}
```

Connections

- Connect Vcc pin in Bluetooth device to 5V or 3.3V in Arduino Board
- Connect GND pin of bluetooth to GND pin on Arduino Board.
- Connect RX pin to pin 9 on Arduino
- Connect TX pin to pin 8 on Arduino
- Connect LED to pin 13 on Arduino.
- Connect the System to Arduino using USB and upload the Code.
- Connect the Bluetooth device to your phone
- Open Serial terminal app on your phone and > Devices > Choose the Bluetooth device.
- The give command '1' to turn on the LED and '0' to turn Off the LED.

OUTPUT


Terminal
15:58:01:757 Connecting to BT6
15:58:09.261 Connected
15:58:09.019 1
15:58:09.225 LED on!
15:58:09.862 0
15:58:13:017 LED off!
15:58:21.376 Connection Lost
M1
M2
M3
M4
M5
M6

VIVA QUESTIONS

1. Define sensors.

Ans. Sensors are devices that detect physical or environmental changes and convert them into signals.

2. What is serial communication in bluetooth?

Ans. Serial communication in Bluetooth is the wireless exchange of data b/w devices using a sequential, bit-by-bit transmission protocol.

3. Name the modules used for bluetooth to connect to arduino.

Ans. Bluetooth modules used to connect to Arduino are HC-05 and HC-06.

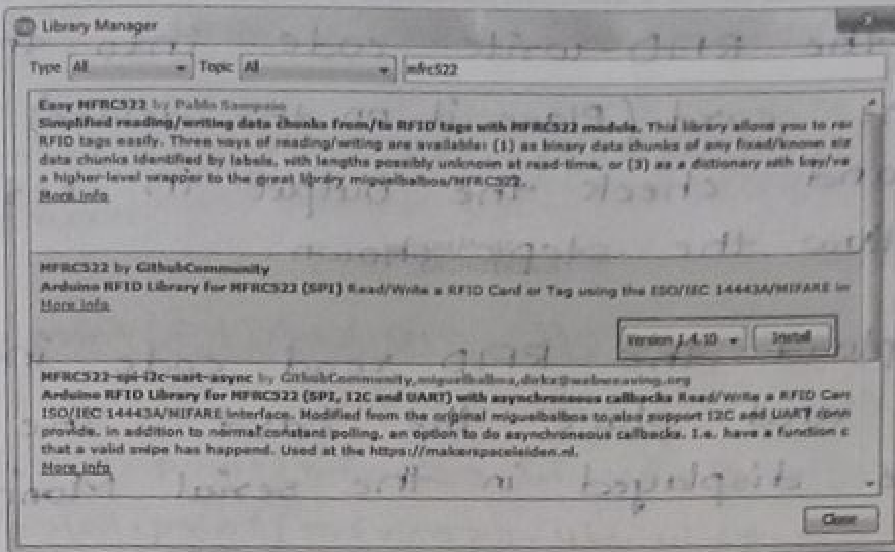
4. Define RX and TX pins?

Ans. RX (Receive) pin receives data, and TX (Transmit) pin sends data between devices.

5. List the type of sensors.

Ans. The type of sensors include, temperature, pressure, smoke/gas sensors, light, humidity, motion sensors.

Gkay
22/1/26



SOURCE CODE

Connections

RFID		Arduino
RST	—	9
MISO	—	12
MOSI	—	11
SDA	—	10
SCK	—	13
GND	—	GND
3.3V	—	3.3V

Steps:-

- Give all the connections.
- In Arduino IDE, go to Sketch > Include Libraries > Manage Libraries > MFRC522.
- From that library open the RFID write code and RFID read code.

- Upload the RFID-write code into the device, Scan the card (Place it on the device) and upload and check the output in serial Monitor. and follow the steps shown.

- Then upload the RFID read code into the device and place the card. The output will be displayed in the serial Monitor.

RFID - write code

```
#include <SPI.h>
#include <MFRC522.h>
#define RST_PIN 9
# define SS_PIN 10
MFRC522 mfrc522(SS_PIN, RST_PIN);
void setup() {
  Serial.begin(9600);
  SPI.begin();
  mfrc522.PCD_Init();
  Serial.println(F("Write personal data on a
              MIFARE PICC"));
}
void loop() {
  MFRC522::MIFARE_Key key;
  for (byte i=0; i<6; i++) key.keyByte[i] = 0xFF;
  if (!mfrc522.PICC_IsNewCardPresent()){
```

```
  return;
}
if (!mfrc522.PICC_ReadCardSerial()) {
    return;
}
Serial.print(F("Card UID:"));
for (byte i=0; i< mfrc522.uid.size; i++) {
    Serial.print(mfrc522.uid.uidByte[i] < 0x10 ? "0" : " ");
    Serial.print(mfrc522.uid.uidByte[i], HEX);
}
Serial.print(F("PICC type: "));
MFRC522::PICC_TYPE piccType = mfrc522.PICC_GetType
                              (mfrc522.uid.sak);
Serial.println(mfrc522.PICC_GetTypeName(piccType));
byte buffer[34];
byte block;
MFRC522::StatusCode status;
byte len;

Serial.setTimeout(20000L);
Serial.println(F("Type Family name, ending with #"));
len = Serial.readBytesUntil('#', (char *) buffer, 30);
for (byte i=len; i<30; i++) buffer[i] = ' ';

block=1;
Status= mfrc522.PCD_Authenticate(MFRC522::PICC_
        CMD_MF_AUTH_KEY_A, block, &key, &(mfrc522.uid));
if (status != MFRC522::STATUS_OK) {
    Serial.print(F("PCD_Authenticate() failed: "));
```

```
  Serial.println(mfrc522.GetStatusCodeName(status));
  return;
}
else Serial.println(F("PCD_Authenticate() success:"));
status= mfrc522.MIFARE_Write(block, buffer, 16);
if(status != MFRC522::STATUS_OK){
   Serial.print(F("MIFARE_Write() failed:"));
   Serial.println(mfrc522.GetStatusCodeName(status));
   return;
}
else Serial.println(F("MIFARE_Write() success:"));
block=2;
```

OUTPUT

Output for write

Card UID: 2C 43 0B 33 PICC type: MIFARE 1KB
Type Family name, ending with #

give some name, example reddy#

PCD_Authenticate() success:
MIFARE_write() success:
MIFARE_Write() success:
Type First name, ending with # sriya#
MIFARE_Write() success:
MIFARE_Write() success:

```
status= mfrc522.PCD_Authenticate (MFRC522:: PICC_CMD_
        MF_AUTH_KEY_A, block, &key, &(mfrc522.uid));
if(status != MFRC522:: STATUS_OK) {
  Serial.print(F("PCD_Authenticate() failed: "));
  serial.println(mfrc522. GetStatusCodeName(status));
  return;
 }
status = mfrc522.MIFARE_Write(block, &buffer [16], 16);
 if(status != MFRC522::STATUS_OK) {
     Serial.print(F("MIFARE_Write() failed: "));
     Serial.println(mfrc522.GetStatusCodeName(status));
     return;
 }
  else  Serial.println(F("MIFARE_Write() success: "));
 Serial.println(F("Type First name, ending with #"));
 len = Serial.readBytesUntil('#', (char *) buffer, 20);
 for(byte i=len; i<20; i++) buffer[i]=' ';
  block = 4;
  status = mfrc522.PCD_Authenticate(MFRC522::PICC_
          CMD_MF_AUTH_KEY_A, block, &key, &(mfrc522.
                                          uid));
 if(status != MFRC522::STATUS_OK){
    Serial.print(F("PCD_Authenticate() failed: "));
    Serial.println (mfrc522.GetStatus CodeName(status));
    return;
  }
  status = mfrc522.MIFARE_Write(block, buffer, 16);
  if(status != MFRC522::STATUS_OK) {
    Serial.print(F(" MIFARE_Write() failed: "));
```

```
Serial.println(mfrc522.GetStatusCodeName(status));
return;
}
else Serial.println(F("MIFARE_Write() success: "));
block = 5;
Status = mfrc522.PCD_Authenticate(MFRC522::PICC_
        CMD_MF_AUTH_KEY_A, block, &key, &(mfrc522
            uid));
if(status != MFRC522::STATUS_OK){
  Serial.print(F("PCD_Authenticate() failed: "));
  Serial.println(mfrc522.GetStatusCodeName(status))
  return;
 }
 status = mfrc522.MIFARE_Write(block, &buffer[16], 16)
 if(status != MFRC522::STATUS_OK){
   Serial.print(F("MIFARE_Write() failed: "));
   Serial.println(mfrc522.GetStatusCodeName(status)
   return;
 }
 else  Serial.println(F("MIFARE_Write() success; ")
 Serial.println(" ");
 mfrc522.PICC_HaltA();
 mfrc522.PCD_StopCrypto1();
 }
```

RFID- Read Code

```
#include <SPI.h>
#include <MFRC522.h>
#define RST_PIN 9
#define SS_PIN 10
MFRC522 mfrc522(SS_PIN, RST_PIN);
void setup(){
  Serial.begin(9600);
  SPI.begin();
  mfrc522.PCD_Init();
  Serial.println(F("Read personal data on a MIFARE
                                          PICC:"));
}
void loop() {
  MFRC522::MIFARE_Key key;
  for(byte i=0; i<6; i++) key.keyByte[i] = 0xFF;
  byte block;
  byte len;
  MFRC522::StatusCode status;
  if (!mfrc522.PICC_IsNewCardPresent()){
    return;
  }
  if (!mfrc522.PICC_ReadCardSerial() {
    return;
  }
  Serial.println(F("**Card Detected:**"));
  mfrc522.PICC_DumpDetailsToSerial(&(mfrc522.uid));
  Serial.print(F("Name: "));
  byte buffer1[18];
  block = 4;
```

```
len=18;
~~sta~~
status = mfrc522.PCD_Authenticate(MFRC522::PICC_
        CMD_MF_AUTH_KEY_A, 4, &key, &(mfrc522.uid));
if(status != MFRC522::STATUS_OK){
    Serial.print(F("Authentication failed:"));
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
}
status = mfrc522.MIFARE_Read(block, buffer1, &len
if(status != MFRC522::STATUS_OK){
    Serial.print(F("Reading failed:"));
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
}
for(uint8_t i=0; i<16; i++)
{
    if (buffer[i] != 32)
    {
        Serial.write(buffer1[i]);
    }
}
Serial.print(" ");
byte buffer2[18];
block=1;
status= mfrc522.PCD_Authenticate(MFRC522::PICC_CMD_
        MF_AUTH_KEY_A, 1, &key, &(mfrc522.uid));
if(status != MFRC522::STATUS_OK){
    Serial.print(F("Authentication failed:"));
    Serial.println(mfrc522.GetStatusCodeName(status));
    return; }
```

```
tus = mfrc522.MIFARE_Read(block, buffer2, &len);
atus != MFRC522::STATUS_OK) {
Serial.print(F("Reading failed: "));
Serial.println(mfrc522.GetStatusCodeName(status));
return;
}
for(uint8_t i=0; i<16; i++){
  Serial.write(buffer2[i]);
}
Serial.println(F("\n**End Reading**\n"));
delay(1000);
mfrc522.PICC_HaltA();
mfrc522.PCD_StopCrypto1();
}.
```

ad Output

End Reading:

* Card Detected:**

ard UID: 2C 43 0B 33

ard SAK: 08

PICC type: MIFARE 1KB

Name:

sriya reddy

**End Reading.

VIVA QUESTIONS

1. What is RFID?

Ans. RFID is a technology used to identify and track objects wirelessly using radio waves.

2. How does an RFID system work?

Ans. An RFID system works by using radio waves to transmit data from an RFID tag to a reader, which then process the information

3. What are some of the main advantages of using RFID?

Ans. RFID enables fast, contactless, identification, improves tracking accuracy, and reduces manual effort.

4. What are some examples of real-world applications that use RFID?

Ans. RFID is used in inventory management, access control systems, contactless payments and animal tracking.

5. What are some of the disadvantages of using RFID?

Ans. RFID can be expensive to implement, may face interface issues, and raises privacy and security concerns

19/2/26

Week - 5

Date: 29/1/26

AIM

Write a program to monitor temperature and humidity using Arduino.

HARDWARE REQUIREMENTS

1. Arduino UNO Board
2. DHT11/DHT22 Temperature and Humidity Sensor
3. Jumper Wires
4. Bread Board

PROCEDURE

1. Connect pin 1 (on the left) of the sensor to +5V.
2. Connect pin 2 of the sensor to whatever your DHTPIN.
3. Connect pin 4 (on the right) of the sensor to GROUND.

SOURCE CODE

```
#include <DHT.h>
#define DHTPIN 2
#define DHTTYPE DHT11 or DHT 22
DHT dht(DHTPIN, DHTTYPE);
void setup(){
    Serial.begin(9600);
```

```
Serial.println(F("Failed to read from DHT test!"));
dht.begin();

}
void loop(){
    delay(2000);
    float h= dht.readHumidity();
    float t = dht.readTemperature
    if (isnan(h) || isnan(t)) {
        Serial.println(F("Failed to read from
                    DHT sensor!"));
        return;
    }
    Serial.print(F("Humidity: "));
    Serial.print(h);
    Serial.print(F("% Temperature: "));
    Serial.print(t);
}
```

Connections

DHT 11		Arduino
VCC	—	3.3V
GND	—	GND
Data	—	2

(or)

DHT 22		Arduino
+	—	3V/5V
-	—	GND
Out	—	2

- Make the connections.
- Also download the DHT11 or DHT Sensor Library

  Sketch > Include Library > Choose.

- Verify the code.
- Upload the code
- Check the output in serial Monitor.

OUTPUT

Humidity: 99.90% Temperature: 26.90
Humidity: 99.90% Temperature: 26.70
Humidity: 99.90% Temperature: 26.50

VIVA QUESTIONS

1. How much voltage is used for DHT11 sensor?

Ans. The DHT11 Sensor operates on 3.3V to 5V DC.

2. What is DHT11 sensor?

Ans. The DHT11 sensor is a digital sensor used to measure temperature and humidity

3. What is the principle used for DHT11 sensor?

Ans. The DHT11 sensor works on the principle of a capacitive humidity sensor and a thermistor for temperature measurement.

4. How does Arduino reads temperature using DHT11?

Ans. Arduino reads temperature from the DHT11 by receiving digitally encoded data from the sensor's internal thermistor via a single-wire communication protocol

5. Name some real time applications for temperature sensor.

Ans. Temperature sensors are used in weather monitoring, HVAC systems, medical equipment, industrial process control, and smart home automation.

29/1/26

5. **SSID and password of your WIFI** connection should be given in source code
6. Compile and upload the program and verify the Sensor readings in serialmonitor
7. Go to cloud application(Thing speak) and Verify values in graph.

Hardware Requirements

1. Node MCU ESP8266 Board
2. IR sensor
3. Jumper wires
4. Bread board
5. WIFI Network

Library manager
→ Thingspeak
> ESP8266.

Secrets.h

#define SECRET_SSID "xxxxxxxxxxxxxxx"

#define SECRET_PASS "xxxxxxxxxx"

#define SECRET_CH_ID xxxxxxxxxxx

#define SECRET_WRITE_APIKEY "xxxxxxxxxxxxxxx"

SOURCE CODE

```
#include <ESP8266WiFi.h>
String apikey = "Q90FYM7L9LSQM4GQ"; //write API key
const char *ssid = "225-20 1896"; // wifi name
const char *pass = "00000000"; //password wifi
const char* server = "api.thingspeak.com";
#define IRPin D4
WiFiClient client;
int value;
void setup()
{
Serial.begin(115200);
pinMode (IRpin, INPUT);
```

```
delay(1000);
Serial.println("Connecting to");
Serial.println(ssid);
WiFi.begin(ssid, pass);
while(WiFi.status() != WL-CONNECTED)
{
delay(2000);
Serial.println(" ");
Serial.println("WiFi Connected");
}
void loop(){
    value = digitalRead(IRpin);
Serial.println(value);
if(value == 0){
    Serial.print("object detected");
}
else{
    Serial.print("no object detected");
}
if(client.connect(server, 80)){
 String poststr = apikey;
 poststr += "&field=";
 poststr += String(value);
 Poststr += "\r\n\r\n";
client.print("POST /update HTTP/1.1\n");
```

```
client.print("Host : api.thingspeak.com\n");
client.print("Connection: close\n");
client.print("X-THINGSPEAKAPIKEY: "+apikey+"\n");
client.print("Content-Type: application/x-www-form-
                        urlencoded\n");
client.print(poststr.length());
client.print("\n\n");
client.print(poststr);
client.stop();
serial.println("waiting....");
delay(1000);
}
}
```

Serial Monitor:-

15:45:48.231 → Waiting....

15:45:49.256 → 0

15:45:49.256 → Object detected. Waiting

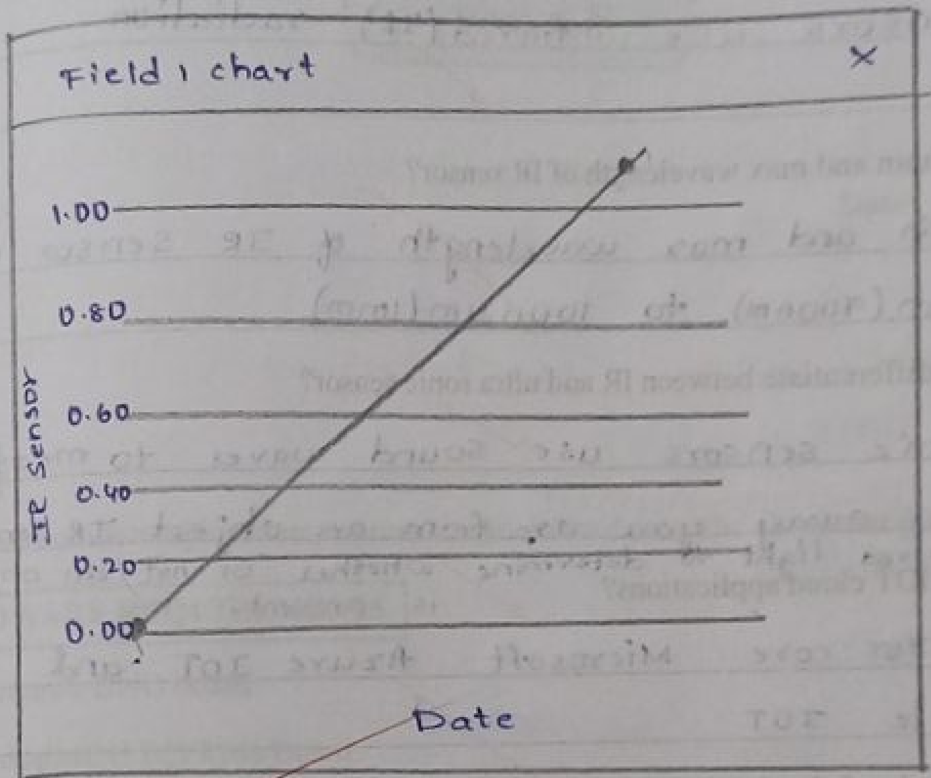
15:45:51.128 → 0

15:45.51.128 → Object deteted waiting

15:45:53.000 → 0

15:45:53.000 → Object Detected waititing

OUTPUT



VIVA QUESTIONS

1. What kind of waves does the IR sensor have?

Ans. IR sensors use infrared (IR) radiation.

2. What is the min and max wavelength of IR sensor?

Ans. The min and max wavelength of IR sensor is 0.7 um (700nm) to 1000 um (1mm)

3. What is the differentiate between IR and ultra sonic sensor?

Ans. Ultrasonic sensors use sound waves to measure how far away you are from an object IR sensors use infrared light to determine whether or not an object is present.

4. Name some IOT cloud applications?

Ans. AWS IOT core, Microsoft Azure IOT and Oracle IOT

5. What is the role of cloud computing in IOT?

Ans. To provide essential, scalable infrastructure for data storage, processing and analytics.

19/2/20

SOURCE CODE

```
#include <DHT.h>
#include <DHT_U.h>
#include <ESP8266WiFi.h>
String apiKey = "ZYUP9R7N1SOEBYRO";
const char *ssid = "xxxx";
const char *pass = "xxxx";
const char* server = "api.thingspeak.com";
#define DHTPIN D3
DHT dht(DHTPIN, DHT11);
WiFiClient client;
void setup()
{
  Serial.begin(115200);
  delay(1000);
  dht.begin();
  Serial.println("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, pass);
  while(WiFi.status() != WL_CONNECTED)
  {
    delay(2000);
    Serial.print(".");
  }
Serial.println(" ");
Serial.println("WiFi connected"); }
```

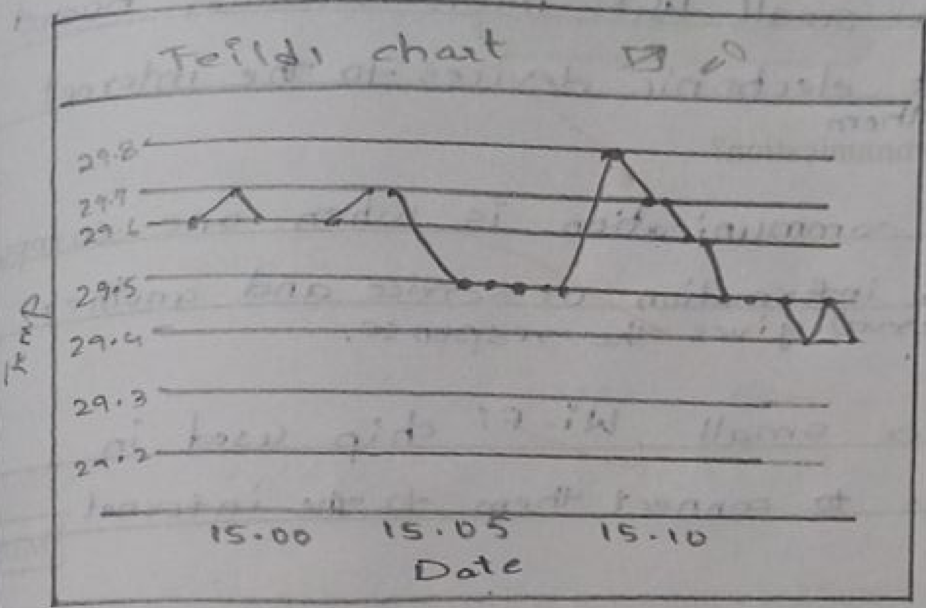
```
void loop()
{
 float h = dht.readHumidity();
 float t = dht.readTemperature();
 if(isnan(h) || isnan(t))
 {
  Serial.println("Failed to read from DHT sensor!");
  return;
 }
 if (client.connect(server,80))
 {
   String postStr = apiKey;
   postStr += "&field1=";
   postStr += String(t);
   postStr += "&field2=";
   postStr += String(h);
   postStr += "\r\n\r\n";
 client.print("POST /update HTTP/1.1\n");
 client.print ("HOst: api.thingspeak.com\n");
 client.print("Connection: close\n");
 client.print("X-THINGSPEAKAPIKEY: "+apikey+"\n");
 client.print("Content-Type: application/x-www-
                  form-urlencoded\n");
 client.print("Content-Length: ");
 client.print(postStr.length());
 client.print("\n\n");
 client.print(postStr);
```

```
Serial.print("Temperature:");
Serial.print(t);
Serial.print("degrees celcius, Humidity:");
Serial.print (h);
Serial.println("%. Send to Thingspeak.");
}
client.stop();
Serial.println ("Waiting....");
delay(1000);
}
}.
```

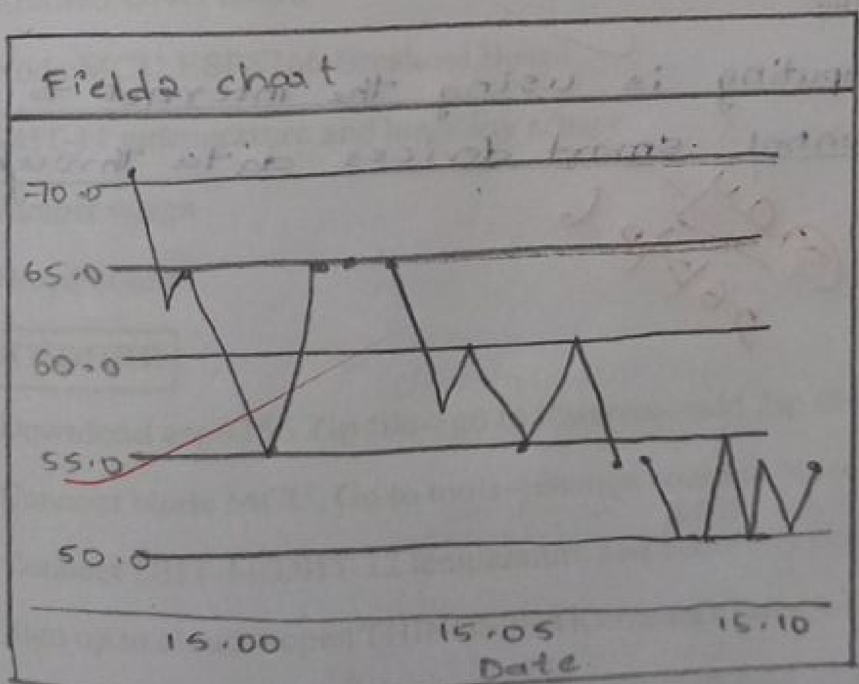
Output in Serial Monitor

Temperature : 29.30 degree celcius , Humidity : 50.00%.
send to thingspeak
Waiting....
Temperature. 29.30 degree celcius , Humidity: 51.00%
send to thingspeak
Waiting...

OUTPUT



Feild1 chart
29.8
29.7
29.6
29.5
29.4
29.3
29.2
15.00
15.05
15.10
Date



Field2 chart
70.0
65.0
60.0
55.0
50.0
15.00
15.05
15.10
Date

VIVA QUESTIONS

1. Define a Node MCU?

Ans. Node MCU is a small WiFi micro-controller board used to connect electronic devices to the internet and control them

2. What is a client-server communication?

Ans. Client-server communication is when one computer (client) asks for information or service and another computer (server) gives the response.

3. What is a ESP8266?

Ans. ESP8266 is a small Wi-Fi chip used in electronic devices to connect them to the internet.

4. What is API key?

Ans. An API key is a secret code used to identify and allow a program to access a service or data securely

5. Define IOT Cloud computing.

Ans. IOT cloud computing is using the internet to store manage and control smart devices data through cloud server.

26/7/26

SOURCE CODE

```
#include "Thingspeak.h"
#include <ESP8266WiFi.h>
#include <DHT.h>
const char ssid[] = "xxx";
const char pass[] = "xxx";
int statusCode = 0;
WiFiClient client;

unsigned long counterChannelNumber = 2846266; //Channel ID
const char* myCounterReadAPIKey = "5XJMCVLJWIZM"; //Read API Key
const int FieldNumber1 = 1;
const int FieldNumber2 = 2;

void setup()
{
Serial.begin(115200);
WiFi.mode(WIFI_STA);
ThingSpeak.begin(client);
}

void loop()
{
  if(WiFi.status() != WL CONNECTED)
  {
    Serial.print("Connecting to ");
    Serial.print(ssid);
    Serial.println(".....");
   while(WiFi.status() != WL_CONNECTED)
   {
    WiFi.begin(ssid, pass);
```

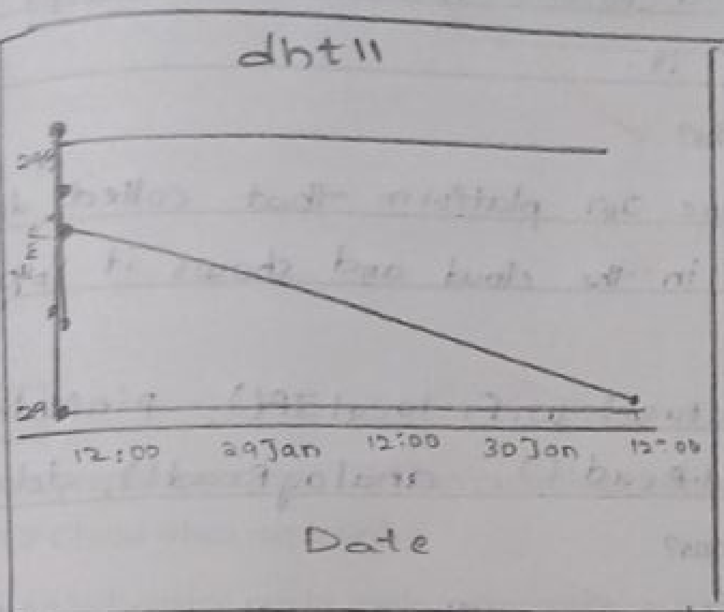
```
delay(5000);
}
Serial.println("Connected to WiFi Successfully.");
}
long temp = ThingSpeak.readLongField(counterChannel
                    Number, FieldNumber1,
   myCounterReadAPIKey);
StatusCode = ThingSpeak.getLastReadStatus();
if (statusCode == 200)
 {
   Serial.print("Temperature:");
   Serial.println(temp);
 }
 else {
   Serial.println("Unable to read channel / No internet
                 Connection");
 }
  delay(100);

long humidity = ThingSpeak.readLongField(counterChannel
            Number, FieldNumber2, myCounterReadAPIkey);
 Status Code = ThingSpeak.getLastReadStatus();
 if (StatusCode == 200)
 {
   Serial.print("Humidity:");
    Serial.println(Humidity);
 }
 else {
    Serial.println("Unable to read channel / No internet
```

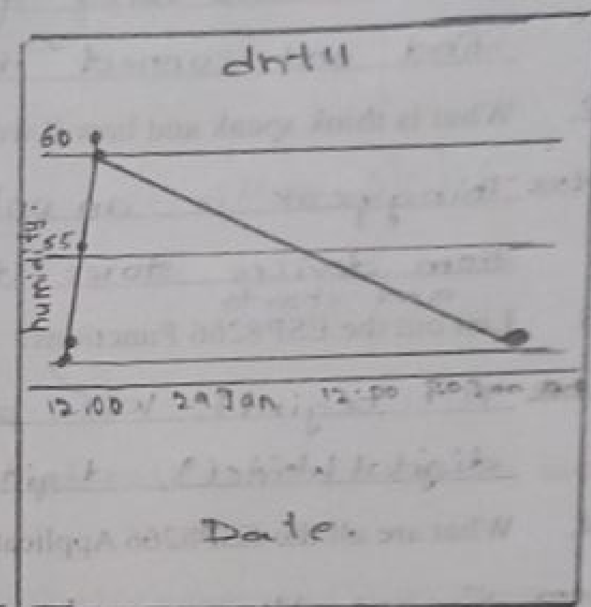
```
    connection");
  }
  delay(100);
}
```

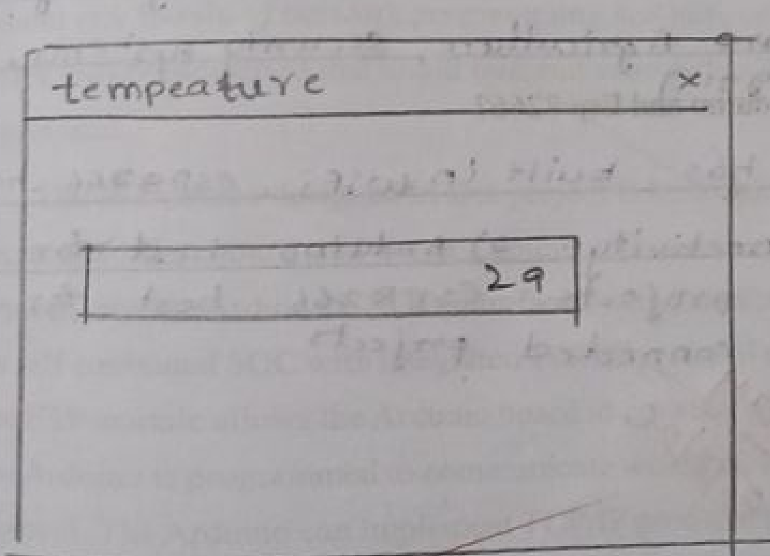
Serial Monitor

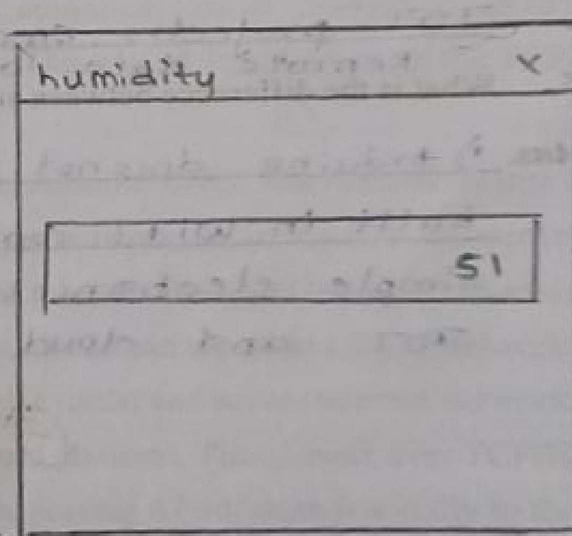
Humidity : 30
Temperature : 29
Humidity : 36
Temperature : 29
Humidity : 36
Temperature : 29

OUTPUT


dht11
29
12:00
29Jan
12:00
30Jan
12:00
Date


dht11
60
55
humidity
12:00
29Jan
12:00
30Jan
Date.


tempeature
×
29


humidity
×
51

VIVA QUESTIONS

1. What is SSID?

Ans. SSID is the name of a Wi-Fi network that helps device find and connect to it.

2. What is think speak and how it works?

Ans. Thingspeak is an online IOT platform that collects data from devices, stores it in the cloud and shows it in graphs and charts

3. List out the ESP8266 Functions?

Ans. WiFi.begin(), WiFi.status(), WiFi.localIP(), pinMode(), digitalWrite(), digital.Read (), analogRead(), delay()

4. What are all the ESP8266 Applications?

Ans. Smart Home automation, weather monitoring system, IOT projects, smart Agriculture, security systems, Remote data logging

5. What is the difference between arduino and Esp 8266?

Ans. 1) Arduino does not has built in WiFi, ESP8266 has built in WiFi connectivity. 2) Arduino best for simple electronics projects. ESP8266 best for IOT and cloud connected projects.

Gray
5/3/26

The DHT11 Temperature and Humidity Sensor is a digital sensor with inbuilt capacitive humidity sensor and Thermistor. It relays a real-time temperature and humidity reading every 2 seconds. The sensor operates on 3.5 to 5.5 V supply and can read temperature between 0° C and 50° C and relative humidity between 20% and 95%. The DHT11 detects water vapors by measuring the electrical resistance between the two electrodes. The humidity sensing component is a moisture holding substrate with electrodes applied to the surface. When water vapors are absorbed by the substrate, ions are released by the substrate which increases the conductivity between the electrodes. The change in resistance between the two electrodes is proportional to the relative humidity. Higher relative humidity decreases the resistance between the electrodes, while lower relative humidity increases the resistance between the electrodes.

SOURCE CODE

```
#include "ESP8266WiFi.h"
#include "DHT.h"
const char* ssid = "sriya";
const char* password = "12345678";
WiFiServer wifiServer(8080);
DHT dht(D3, DHT22);
void setup() {
    Serial.begin (115200);
    delay(1000);
    WiFi.begin (ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.println("Connecting...");
  }
    Serial.print("Connected to WiFi. IP: ");
    Serial.println(WiFi.localIP());
    wifiServer.begin();
    dht.begin();
```

```
}
void loop(){
    WiFiClient  client = wifiServer.available();
    if(client){
        while (client.connected()){
            while(client.available()>0){
                float t=dht.readTemperature();
                float h= dht.readHumidity();
                client.print("humidity:");
                client.print("temperature:");
                client.println(h);
                Serial.println(h);
                client.println(t);
                Serial.println(t);
                delay(2000);
            }
        }
        client.stop();
        Serial.println("Client disconnected");
    }
}
```

Output:-

11:40:57.483 → 33.00

11:40:57.483 → 31.50

11:40:59.479 → 33.00

11:40:59.524 → 31.50

11:41:29.425 → Connecting..

11:41:31.446 → Connecting...

11:41:32.451 → Connected to WiFi. IP: 192.168.10.102

Serial Monitor

11:39:52.585 → 32.00

11:39:52.624 → 31.90

11:39:54.638 → 32.00

11:39:54.638 → 31.80

11:39:56.640 → 32.00

11:39:56.675 → 31.80

11:39:58.691 → 32.00

11:40:08.842 → 31.70.

OUTPUT

TCP Client.

IP address or domain

10.196.68.185

Port

8080

Connect

Disconnect

Received:

humidity:
temperature:
32.00
31.90
humidity:
temperature:
32.00
31.70
humidity.

Clear

☐ HEX ☐ Auto Scroll

Message for send:

hii

☐ Add CR LF

Send message

VIVA QUESTIONS

1. Define TCP?

Ans. A TCP client requests a service, while a TCP server listens and responds to the client over a TCP connection

2. What is TCP Client and server?

Ans. TCP is a reliable, connection. TCP client is a program that initiates a connection and sends request. TCP server is a program that waits for connections and responds to the requests.

3. Explain TCP protocol header format?

Ans. The TCP header format contains fields like source port, Destination port, sequence number, Ack Number, flag, window size, checksum and Urgent pointer.

4. How TCP protocol provides reliability?

Ans. TCP provides reliability using acknowledgements, sequence numbers, transmission and error checking

5. Write down the name of services provided by TCP?

Ans. TCP provides reliable, connection oriented, ordered, and error-checked data delivery services.

G 10ay
12/3/26

SOFTWARE REQUIRED

- ThingSpeak server
- Arduino IDE

ESP8266 board needs to be loaded with the firmware code. In this tutorial, the firmware code is written using Arduino IDE. It is loaded to the ESP8266 board using the Arduino UNO. A generic ESP8266 board is used in this project. This board does not have any bootstrapping resistors, no voltage regulator, no reset circuit and no USB-serial adapter. The ESP8266 module operates on 3.3 V power supply with current greater than or equal to 250 mA. So, CP2102 USB to serial adapter is used to provide 3.3 V voltage with enough current to run ESP8266 reliably in every situation.

The ESP8266 Wi-Fi Module is a self contained SOC with integrated TCP/IP protocol stack that can access to a Wi-Fi network. The ESP8266 is capable of either hosting an application or off loading all Wi-Fi networking functions from another application processor. Each ESP8266 module comes pre-programmed with an AT command set firmware.

The Chip Enable and VCC pins of the module are connected to the 3.3 V DC while Ground pin is connected to the common ground. The chip enable pin is connected to VCC via a 10K pull up resistor. The RESET pin is connected to the ground via a tactile switch where the pin is supplied VCC through a 10K pull up resistor by default. The Tx and Rx pins of the module are connected to the RX and TX pins of the Arduino UNO. The GPIO-0 pin of the module is connected to ground via a tactile switch where the pin is supplied VCC through a 10K pull up resistor by default.

SOURCE CODE

```
#include <ESP8266WiFi.h>
#include <WiFiUdp.h>
#include <DHT.h>
const char* ssid = "Sriya";
const char* password = "12345678";
const char* udpAddress = "192.168.87.225";
```

```
const int udpPort= 1234;
#define DHTPIN D3
# define DHTTYPE DHT22.
DHT dht (DHTPIN, DHTTYPE);
WiFiUDP udp;
void setup() {
    Serial.begin(115200);
    Serial.println();
    Serial.println("Connecting to WiFi...");
    WiFi.begin (ssid, password);
    while(WiFi.status()!= WL_CONNECTED){
        delay(2000);
        Serial.print(".");
    }
    Serial.println();
    Serial.println("WiFi connected");
    dht.begin();
}
void loop(){
    delay(2000);
    float temperature = dht.readTemperature();
    float humidity= dht.readHumidity();
    if (isnan(temperature)|| isnan(humidity)){
        Serial.println ("Failed to, read from DHT
                                    sensor!");
        return;
    }
```

```
Serial.print("Temperature : ");
Serial.print(temperature);
Serial.print(" °C\t Humidity : ");
Serial.print(humidity);
Serial.println(" %");
Serial.println("sending data over UDP...");
udp.beginPacket(udpAddress, udpPort);
udp.print("Temperature: ");
udp.print(temperature);
udp.print(" °C, Humidity: ");
udp.print(humidity);
udp.println(" %");
udp.endPacket();
Serial.println("Data sent over UDP.");
}
```

Serial Monitor:-

11:48:10.485 → Temperature : 29.90 °C Humidity : 33.00%
11:48:10.485 → Sending data over UDP...
11:48:10.485 → Data sent over UDP.
11:48:20.532 → Temperature: 29.90°C Humidity : 33.00%
11:48:20.532 → Sending data over UDP....
11:48:20.532 → Data sent over UDP.

OUTPUT

(→) Rx = 192.168.137.37:8081
Tx = 192.168.0.7:8081

Rx: 344B, Tx: 0B.

Temperature: 30.40°c, Humidity: 56.00%.
Temperature: 30.40°c, Humidity: 56.00%.
Temperature: 30.40°c, Humidity: 54.00%.
Temperature: 30.40°c, Humidity: 54.00%.
Temperature: 30.50°c, Humidity: 55.00%.
Temperature: 30.50°c, Humidity: 55.00%.

Type command	+	\n	[>]

VIVA QUESTIONS

1. What is a socket in the context of network programming?

Ans. A socket is an endpoint used for sending and receiving data between two devices over a network.

2. What is the primary function of a router?

Ans. The primary function of a router is to forward data packets between different networks.

3. What's the main advantage of using UDP over TCP?

Ans. Faster data transmission with lower overhead.

4. What are uses of UDP.

Ans. UDP is used for fast, real time communication like streaming, online gaming, VOIP, and DNS queries

5. Explain UDP protocol header format?

Ans. The UDP header format consists of four field: Source port, Destination port, length and checksum

12/2/26